
Security Review Report

NM-0615 Panoptic



NETHERMIND
SECURITY

(June 17, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	The HypoVault Contract	4
4.2	The PanopticVaultAccountant	4
4.3	The Manager and Curation Framework	4
4.4	Permissionless Deployment and Fund Management	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[Low] Rounding error in executeDeposit(...) may lead to state inconsistency and excessive share minting	7
6.2	[Low] Rounding error in share distribution leads to locked yield	9
6.3	[Info] Manager has exclusive control over deposit cancellations	10
6.4	[Info] The decimals(...) function returns a misleading value of 0 for underlying tokens that do not implement it	10
7	Documentation Evaluation	11
8	Test Suite Evaluation	12
8.1	Compilation Output	12
8.2	Tests Output	13
9	About Nethermind	16

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Panoptic's HypoVault](#) contracts. The **HypoVault** provides a decentralized asset management framework that allows users to deposit funds into on-chain investment vehicles. These vaults are actively managed by designated strategists who execute trading strategies to generate yield. The protocol uses an asynchronous, epoch-based system for deposits and withdrawals to ensure fair pricing for all participants.

This audit focused on the smart contracts that implement the protocol's core logic, including the **HypoVault** contract, which holds assets and issues shares; the **PanopticVaultAccountant**, responsible for secure Net Asset Value (NAV) calculation; the **HypoVaultFactory**, which enables the permissionless creation of new vaults; and the **HypoVaultManagerWithMerkleVerification**, which facilitates trust-minimized strategy execution.

The audit comprises 690 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report four points of attention, where two are classified as Low, and two are classified as Informational severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

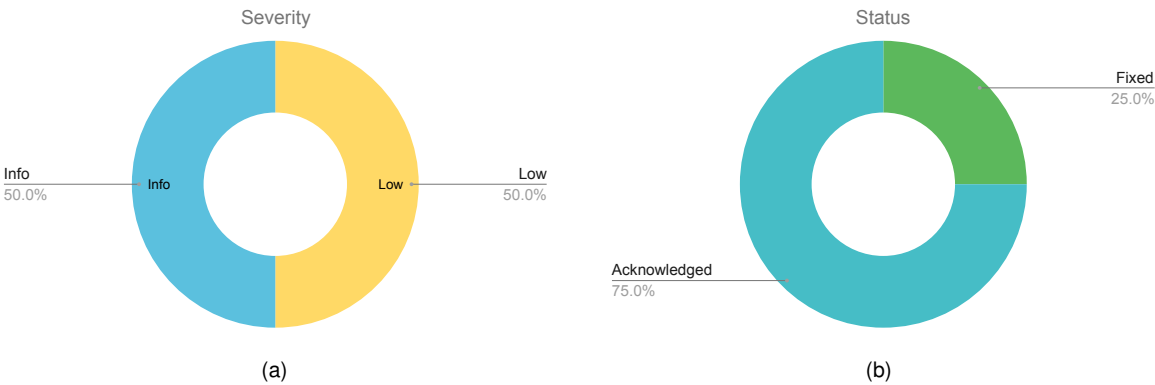


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (2), Undetermined (0), Informational (2), Best Practices (0). Distribution of status: Fixed (1), Acknowledged (3), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	August 25, 2025
Final Report	June 17, 2026
Initial commit	dc89b9ce4f826e0c6ec2ec6dde5234e66ed61577
Final commit	a00f269df24065b3982c27cda5a6905def12e1c8
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/HypoVault.sol	384	189	49.2%	144	717
2	src/HypoVaultFactory.sol	37	24	64.9%	6	67
3	src/managers/HypoVaultManagerWithMerkleVerification.sol	48	24	50.0%	12	84
4	src/interfaces/IVaultAccountant.sol	8	6	75.0%	1	15
5	src/interfaces/IHypoVault.sol	22	2	9.1%	7	31
6	src/accountants/PanopticVaultAccountant.sol	191	54	28.3%	40	285
	Total	690	299	43.3%	210	1199

* The review of the *PanopticVaultAccountant.sol* contract was limited in scope. A complete analysis was not performed, as it would require an in-depth understanding of the core Panoptic protocol, which was outside the scope of this engagement.

3 Summary of Issues

	Finding	Severity	Update
1	Rounding error in executeDeposit(...) may lead to state inconsistency and excessive share minting	Low	Acknowledged
2	Rounding error in share distribution leads to locked yield	Low	Acknowledged
3	Manager has exclusive control over deposit cancellations	Info	Fixed
4	The decimals(...) function returns a misleading value of 0 for underlying tokens that do not implement it	Info	Acknowledged

4 System Overview

The Panoptic Vault protocol provides a framework for decentralized asset management through its core component, the HypoVault. It is designed to separate the roles of capital providers (Depositors) from strategy executors (Managers). Users can deposit assets to gain passive exposure to complex, actively managed trading strategies. The system employs an asynchronous, epoch-based model for deposits and withdrawals to ensure fair pricing and mitigate timing-based advantages. New vaults can be created permissionlessly by anyone using the provided factory contract.

4.1 The HypoVault Contract

The HypoVault is the central contract that serves as both the asset custodian and the on-chain representation of ownership. It functions as an ERC20 token, where each token represents a share of the vault's total assets. The contract is responsible for managing the lifecycle of user funds through a queuing system.

- **Asset Custody:** It holds all assets deposited by users. These funds can only be deployed externally through strictly controlled functions callable by the designated Manager.
- **Epoch Management:** It tracks the state of deposits and withdrawals through a series of epochs. Key state variables like `depositEpoch`, `withdrawalEpoch`, `depositEpochState`, and `withdrawalEpochState` are used to segregate and process user requests in batches.
- **User Queues:** It maintains mappings (`queuedDeposit` and `queuedWithdrawal`) to record individual user requests for each epoch, ensuring that assets and shares are allocated on a pro-rata basis during fulfillment.
- **ERC-20 Considerations:** The contract may not always adhere to the common ERC-20 behavior where `totalSupply()` equals the sum of all individual user balances. This occurs because the logic that updates the `totalSupply` is separate from the logic that mints shares to individual user accounts. For instance, the total supply may be increased in one step to reflect a batch of new deposits, while the corresponding shares are distributed to users in a subsequent, distinct operation.
- **Token Compatibility:** The vault's internal accounting is designed for standard ERC20 tokens. It is not compatible with fee-on-transfer or rebasing tokens, as these can cause discrepancies between the assets transferred and the amount recorded, leading to state inconsistencies. It is also not compatible with tokens with huge balances.

4.2 The PanopticVaultAccountant

The PanopticVaultAccountant is a dedicated, external valuation engine responsible for calculating the vault's Net Asset Value (NAV). This separation of concerns is a critical security feature, as it prevents a potentially malicious Manager from manipulating the vault's reported value. Its primary function, `computeNAV()`, is called during the fulfillment of deposits and withdrawals. The valuation process uses a hybrid model that combines manager-provided data with on-chain verification.

- **Manager-Provided Data:** The Manager submits a payload of data (`managerInput`) containing off-chain information such as a complete list of the vault's positions and current price quotes for the assets.
- **On-Chain Verification:** The accountant contract performs several critical on-chain safety checks before finalizing the NAV calculation:
 - It verifies that the hash of the provided pool information matches a pre-approved configuration set by the Curator, reverting if there is a mismatch.
 - It ensures the list of positions provided by the Manager is complete by comparing it against the vault's actual on-chain holdings.
 - It validates the Manager's price quotes against on-chain Time-Weighted Average Price (TWAP) oracles, reverting if the deviation exceeds a pre-configured threshold.

4.3 The Manager and Curation Framework

The protocol establishes a clear separation of powers between the Manager and the Curator to enhance security and operational integrity.

- **Manager:** The Manager is the active strategist responsible for executing trades via the `manage()` function and processing user fund flows by calling `fulfillDeposits()` and `fulfillWithdrawals()`. A contract such as `HypoVaultManagerWithMerkleVerification` is intended to be set as the vault's manager. This contract acts as a proxy, allowing a Curator to grant restricted, role-based access to multiple strategists. These strategists can only execute actions that are part of a pre-approved Merkle root, enforcing a trust-minimized operational model.
- **Curator:** The Curator is an administrative role, typically the owner of the PanopticVaultAccountant and manager contracts. They are responsible for the initial security setup, defining the approved list of pools the vault can interact with, and setting key risk parameters such as maximum price deviation for oracles.
- **Forced Withdrawals:** The Manager has the authority to initiate a withdrawal on behalf of any user by calling `requestWithdrawalFrom()`. This can be used to eject a user from the vault or to trigger "fee collection" by setting a `shouldRedeposit` flag. In the latter case, the user's accrued profits are realized, performance fees are collected, and the funds are automatically re-deposited into the next epoch.

4.4 Permissionless Deployment and Fund Management

The system is designed to be extensible and follows a structured, asynchronous process for handling user funds.

- **Factory:** The `HypoVaultFactory` contract allows any user to permissionlessly deploy a new, independent `HypoVault` instance. The creator specifies the vault's underlying asset, manager, accountant, and performance fee structure. The factory emits a `VaultCreated` event, which provides an on-chain registry for discovering new vaults.
- **Epoch-Based Flow:** Deposits and withdrawals are not processed instantly but are batched into epochs. This ensures all participants in a given batch receive a uniform price based on a single NAV calculation, protecting them from front-running. The flow for both deposits and withdrawals consists of three distinct phases:
 - a. **Request Phase:** A user initiates a deposit or withdrawal, and their request is added to the queue for the current, unfulfilled epoch.
 - b. **Fulfillment Phase:** The Manager closes the current epoch by calling the appropriate fulfillment function. This triggers the NAV calculation, determines the share-to-asset conversion rate for that epoch, and updates the vault's total supply.
 - c. **Execution Phase:** After an epoch is fulfilled, users must call an execution function (`executeDeposit()` or `executeWithdrawal()`) to claim their corresponding shares or assets. Any portion of their request that was not fulfilled by the Manager is automatically rolled over to the next epoch's queue.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Rounding error in executeDeposit(...) may lead to state inconsistency and excessive share minting

File(s): `src/HypoVault.sol`

Description: The vault's deposit mechanism is spread across three functions: `requestDeposit(...)`, where users queue their assets; `fulfillDeposits(...)`, where the manager finalizes an epoch by specifying the total assets to be invested; and `executeDeposit(...)`, where users claim their shares based on the fulfilled amount.

In the `executeDeposit(...)` function, the portion of a user's deposit that gets fulfilled (`userAssetsDeposited`) is calculated pro-rata using `Math.mulDiv(...)`. Any of the user's assets that are not fulfilled (`assetsRemaining`) are carried over to the next deposit epoch.

```

1  function executeDeposit(address user, uint256 epoch) external {
2      if (epoch >= depositEpoch) revert EpochNotFulfilled();
3      uint256 queuedDepositAmount = queuedDeposit[user][epoch];
4      queuedDeposit[user][epoch] = 0;
5
6      DepositEpochState memory _depositEpochState = depositEpochState[epoch];
7      // @audit-issue Due to downward rounding, the sum of all `userAssetsDeposited`
8      // across all users will be less than `_depositEpochState.assetsFulfilled`.
9      uint256 userAssetsDeposited = Math.mulDiv(
10         queuedDepositAmount,
11         _depositEpochState.assetsFulfilled,
12         _depositEpochState.assetsDeposited
13     );
14     // ...
15     // @audit The unfulfilled portion of the deposit is carried over to the user's balance in the next epoch.
16     uint256 assetsRemaining = queuedDepositAmount - userAssetsDeposited;
17     if (assetsRemaining > 0) queuedDeposit[user][epoch + 1] += uint128(assetsRemaining);
18     // ...
19 }

```

The issue is that `Math.mulDiv(...)` rounds down. Consequently, the sum of all `userAssetsDeposited` across all depositors in an epoch will be slightly less than the total `assetsFulfilled` defined by the manager.

This dust is problematic because while the `assetsRemaining` for each user (including their share of the dust) is carried over to the next epoch in the `queuedDeposit` mapping, the global state is not updated correspondingly. In `fulfillDeposits(...)`, the next epoch's total `assetsDeposited` is calculated simply as the total unfulfilled assets, without accounting for the accumulated user-level dust from the previous epoch's rounding errors.

```

1  function fulfillDeposits(...) external onlyManager {
2      // ...
3      // @audit The global remaining assets are calculated here.
4      uint256 assetsRemaining = epochState.assetsDeposited - assetsToFulfill;
5      // ...
6      currentEpoch++;
7      depositEpoch = uint128(currentEpoch);
8      // @audit The next epoch's global deposit amount is set. This value will be smaller than the
9      // sum of all users' queued deposits due to the rounding errors from the prior epoch.
10     depositEpochState[currentEpoch] = DepositEpochState({
11         assetsDeposited: uint128(assetsRemaining),
12         sharesReceived: 0,
13         assetsFulfilled: 0
14     });
15     // ...
16 }

```

This creates a state inconsistency where the sum of individual queued deposits for an epoch is greater than the globally tracked `depositEpochState[epoch].assetsDeposited`. This inconsistency can cause an excessive number of shares to be minted. In the subsequent epoch, the denominator for the pro-rata calculation (`_depositEpochState.assetsDeposited`) will be smaller than the actual sum of queued assets (the numerator). This inflates the fulfillment ratio, causing more shares to be minted than are backed by assets and diluting the value of all existing shareholders in the vault.

Recommendation(s): Consider modifying the current design to avoid this or consistently monitor the state of vaults to act quickly before the accumulation of this dust can become a problem.

Status: Acknowledged.

Update from the client: The underlying cause is that partial deposit fulfillments round down per user, while the next epoch's global deposited amount only carries forward the aggregate remainder. This creates a mismatch between user-level queued deposits and the global epoch denominator. That said, the practical impact is limited to dust-level drift in the underlying asset's smallest unit. For assets like USDC, the maximum mismatch per partially fulfilled epoch is bounded by the number of depositors minus one base unit, making economically meaningful extraction infeasible on Ethereum mainnet absent an unrealistic number of sybil depositors. We therefore acknowledge the issue as a real accounting edge case, but do not view it as a credible loss-of-funds or protocol-bricking risk under expected operating conditions.

A concrete USDC example shows the scale. If three users each deposit 1 USDC and only 2 USDC is fulfilled, each user is credited 0.666666 USDC and rolls 0.333334 USDC. The true rollover total is 1.000002 USDC, while the vault globally records 1.000000 USDC, creating a mismatch of only 0.000002 USDC total. If the same cohort is fully processed on the next fulfillment, that dust offsets exactly across that same cohort. A transfer only appears if the shareholder set changes between the partial fulfillment and the later rollover settlement, such as if new users deposit in between.

The magnitude is extremely small in native asset units. For USDC, this example is 2×10^{-6} USDC, which is six orders of magnitude below 1 USDC. For WETH, an analogous two-unit mismatch would be 2×10^{-18} WETH, which is eighteen orders of magnitude below 1 WETH. More generally, the drift is bounded by the number of depositors minus one in base units, so the impact depends on the number of distinct depositor addresses in a partially fulfilled epoch, not on the raw share-unit count.

From an attack perspective, an attacker would need many distinct depositor addresses all participating in a partially fulfilled epoch in order to accumulate any measurable benefit, and even then the result would only be dust. If assuming the vault managers fulfill on a regular cadence, the expected small depositor count per epoch, and the fact that partial fulfillment only occurs when the cap is hit and is then resolved on the next cycle, such an attack is extremely unlikely and economically unprofitable on Ethereum mainnet.

6.2 [Low] Rounding error in share distribution leads to locked yield

File(s): `src/HypoVault.sol`

Description: In the vault's deposit lifecycle, the `fulfillDeposits(...)` function calculates the total number of new shares to be created for an epoch and increases the contract's `totalSupply` accordingly. Subsequently, depositors call the `executeDeposit(...)` function to mint their individual portion of these new shares.

A problem arises in the `executeDeposit(...)` function where individual share amounts are calculated. The calculation uses `Math.mulDiv(...)`, which rounds down.

```

1  function executeDeposit(address user, uint256 epoch) external {
2      // ...
3      // @audit-issue The downward rounding here means the sum of shares received by all
4      // users can be less than the total shares allocated for the epoch.
5      uint256 sharesReceived = Math.mulDiv(
6          userAssetsDeposited,
7          _depositEpochState.sharesReceived,
8          _depositEpochState.assetsFulfilled == 0 ? 1 : _depositEpochState.assetsFulfilled
9      );
10
11     _mintVirtual(user, sharesReceived);
12     // ...
13 }

```

Due to this downward rounding, the sum of all shares minted to individual users throughout an epoch may be less than the total amount by which the `totalSupply` was increased in `fulfillDeposits(...)`.

```

1  function fulfillDeposits(...) external onlyManager {
2      // ...
3      uint256 _totalSupply = totalSupply;
4      uint256 sharesReceived = Math.mulDiv(assetsToFulfill, _totalSupply, totalAssets);
5      // ...
6      // @audit The totalSupply is increased by the full calculated amount,
7      // before individual (rounded-down) shares are minted to users.
8      totalSupply = _totalSupply + sharesReceived;
9      emit DepositsFulfilled(currentEpoch - 1, assetsToFulfill, sharesReceived);
10 }

```

This creates a persistent discrepancy where the `totalSupply` is greater than the sum of all user balances. The resulting difference is a growing number of shares that are not owned by any address.

Since the vault's Net Asset Value (NAV) and any generated profits are divided across the entire `totalSupply`, these shares are entitled to a portion of the vault's assets. This portion becomes locked in the contract, as there is no owner who can claim or redeem these shares. The portion of locked funds will growth with every new deposit epoch, continuously eroding the returns for legitimate shareholders.

Recommendation(s): Consider implementing a mechanism to handle the rounding dust from the share calculation in `executeDeposit(...)`. For example, the last user to execute their deposit in an epoch receives any remaining fractional shares, thus aligning the sum of minted balances with the increase in `totalSupply`.

Status: Acknowledged.

Update from the client: The vault increases total supply for the full batch at fulfillment time, while per-user share allocations during execution are rounded down. This can leave a very small number of unowned shares in the system, which slightly dilutes returns over time. In practice this is a dust-level effect and not realistically exploitable for meaningful value extraction. We acknowledge the issue, but view the impact as negligible.

6.3 [Info] Manager has exclusive control over deposit cancellations

File(s): `src/HypoVault.sol`

Description: When users deposit assets into the HypoVault via the `requestDeposit(...)` function, their tokens are immediately transferred to and held by the contract. These funds remain in a pending state until processed by the vault's manager.

The contract provides a `cancelDeposit(...)` function to return these pending assets to the user. However, this function includes an `onlyManager` modifier, making the manager the only entity with the authority to execute a cancellation.

```

1 // @audit Only the manager can cancel a deposit.
2 function cancelDeposit(address depositor) external onlyManager {
3     uint256 currentEpoch = depositEpoch;
4     uint256 queuedDepositAmount = queuedDeposit[depositor][currentEpoch];
5     queuedDeposit[depositor][currentEpoch] = 0;
6
7     depositEpochState[currentEpoch].assetsDeposited -= uint128(queuedDepositAmount);
8
9     SafeTransferLib.safeTransfer(underlyingToken, depositor, queuedDepositAmount);
10
11     emit DepositCancelled(depositor, queuedDepositAmount);
12 }

```

This design places complete control over pending deposits with the manager role. Users cannot unilaterally retrieve their funds after depositing them. The return of unfulfilled assets is contingent upon the manager's action, creating a strong trust assumption regarding the manager's liveness and integrity.

Recommendation(s): The current design centralizes control over pending user funds. For future consideration, the team might evaluate introducing a permissionless way for users to cancel their own deposits.

Status: Fixed.

Update from the client: Fixed in commit [a00f26](#). Users can now cancel their own pending deposits via the permissionless `cancelDeposit(...)` path, so the trust/liveness concern identified in the audit has been addressed.

6.4 [Info] The `decimals(...)` function returns a misleading value of 0 for underlying tokens that do not implement it

File(s): `src/HypoVault.sol`

Description: The HypoVault contract implements the ERC20 standard, representing shares in the vault. Its `decimals(...)` function is designed to mirror the decimals of the `underlyingToken` to ensure consistency.

The current implementation uses a `try...catch` statement to fetch the decimals from the underlying token. If the call to `underlyingToken.decimals()` fails—for instance, if the token contract does not implement this function—the catch block returns 0.

```

1 function decimals() external view returns (uint8) {
2     try IERC20Metadata(underlyingToken).decimals() returns (uint8 _decimals) {
3         return _decimals;
4     } catch {
5         // @audit-issue If the underlying token does not implement decimals(), this returns 0.
6         return 0;
7     }
8 }

```

The `decimals()` function is an optional part of the ERC20 standard. While a return value of 0 is valid and indicates a non-divisible token.

Returning 0 when the function is not implemented is misleading and can cause integration issues.

Recommendation(s): Consider modifying the behavior for underlying tokens that do not implement the `decimals()` function. Instead of returning 0, a different approach could be to revert the transaction to make the incompatibility explicit or set the decimals during initialization.

Status: Acknowledged.

Update from the client: This behavior may be misleading for integrators or user interfaces interacting with non-standard ERC20 tokens, but it does not create an onchain loss-of-funds scenario. We acknowledge the issue as a compatibility/documentation concern rather than a material protocol risk.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Panoptic's documentation

The Panoptic team provided comprehensive documentation that clearly outlined the system's architecture, core components, and intended functionalities. The codebase itself is well-documented, which greatly facilitated the audit process. Furthermore, the team promptly and thoroughly addressed all questions raised by the Nethermind Security team, providing valuable insights and a deeper understanding of the protocol's technical aspects.

8 Test Suite Evaluation

8.1 Compilation Output

```
> forge compile
[] Compiling...
[] Compiling 111 files with Solc 0.8.28
[] Solc 0.8.28 finished in 49.05s
Compiler run successful with warnings:
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:2547:10:
|
|      (uint128 assetsDeposited1, uint128 sharesReceived1, uint128 assetsFulfilled1) = vault
|      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:2547:36:
|
|      (uint128 assetsDeposited1, uint128 sharesReceived1, uint128 assetsFulfilled1) = vault
|      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:2547:61:
|
|      (uint128 assetsDeposited1, uint128 sharesReceived1, uint128 assetsFulfilled1) = vault
|      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3373:9:
|
|      uint256 bobShares = vault.balanceOf(Bob);
|      ^^^^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3494:10:
|
|      (uint128 sharesWithdrawnBefore, , ) = vault.withdrawalEpochState(1);
|      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3526:10:
|
|      (uint128 amount, uint128 basis, bool shouldRedeposit) = vault.queuedWithdrawal(Alice, 0);
|      ^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3526:26:
|
|      (uint128 amount, uint128 basis, bool shouldRedeposit) = vault.queuedWithdrawal(Alice, 0);
|      ^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3564:27:
|
|      (uint128 amount1, uint128 basis1, bool shouldRedeposit1) = vault.queuedWithdrawal(Alice, 0);
|      ^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3573:27:
|
|      (uint128 amount2, uint128 basis2, bool shouldRedeposit2) = vault.queuedWithdrawal(Alice, 0);
|      ^^^^^^^^^^^^^^
|
Warning (2072): Unused local variable.
--> test/HypoVault.t.sol:3583:27:
|
|      (uint128 amount3, uint128 basis3, bool shouldRedeposit3) = vault.queuedWithdrawal(Alice, 0);
|      ^^^^^^^^^^^^^^
```

8.2 Tests Output

```
> forge test
Ran 23 tests for test/HypoVaultManagerWithMerkleVerification.t.sol:HypovaultManagerWithMerkleVerificationTest
[PASS] testDeploymentFlow() (gas: 34914)
[PASS] test_all_functions_require_authorization() (gas: 42915)
[PASS] test_cancelDeposit_multiple_users() (gas: 133219)
[PASS] test_cancelDeposit_success() (gas: 94844)
[PASS] test_cancelWithdrawal_success() (gas: 263618)
[PASS] test_complete_deposit_withdrawal_cycle() (gas: 372450)
[PASS] test_fulfillDeposits_partial() (gas: 278761)
[PASS] test_fulfillDeposits_success() (gas: 243650)
[PASS] test_fulfillDeposits_with_manager_input() (gas: 226889)
[PASS] test_fulfillWithdrawals_max_assets_protection() (gas: 258792)
[PASS] test_fulfillWithdrawals_partial() (gas: 336044)
[PASS] test_fulfillWithdrawals_success() (gas: 309286)
[PASS] test_fulfill_with_zero_amounts() (gas: 97189)
[PASS] test_multiple_strategist_management() (gas: 337263)
[PASS] test_onlyStrategist_modifier_allows_owner() (gas: 89669)
[PASS] test_onlyStrategist_modifier_allows_strategist() (gas: 88476)
[PASS] test_onlyStrategist_modifier_reverts_for_unauthorized() (gas: 107374)
[PASS] test_onlyStrategist_modifier_reverts_for_zero_root() (gas: 105217)
[PASS] test_owner_can_remove_strategist_access() (gas: 163356)
[PASS] test_requestWithdrawalFrom_with_redeposit() (gas: 313816)
[PASS] test_requestWithdrawalFrom_without_redeposit() (gas: 293828)
[PASS] test_setManageRoot_only_owner() (gas: 115576)
[PASS] test_zero_amount_operations() (gas: 271648)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 11.50ms (3.72ms CPU time)

Ran 92 tests for test/HypoVault.t.sol:HypoVaultTest
[PASS] test_basis_calculation_precision() (gas: 247221)
[PASS] test_basis_transfer_conservation_with_min_logic() (gas: 313765)
[PASS] test_basis_transfer_min_logic_actually_limits_transfer() (gas: 385501)
[PASS] test_basis_transfer_min_logic_prevents_basis_inflation() (gas: 367995)
[PASS] test_basis_transfer_min_logic_with_zero_recipient_balance() (gas: 316065)
[PASS] test_basis_transfer_multiple_transfers() (gas: 338359)
[PASS] test_basis_transfer_new_min_logic() (gas: 365811)
[PASS] test_basis_transfer_on_share_transfer() (gas: 291635)
[PASS] test_basis_transfer_on_transferFrom() (gas: 319232)
[PASS] test_basis_transfer_precision() (gas: 267972)
[PASS] test_basis_transfer_with_different_basis_per_share_ratios() (gas: 375129)
[PASS] test_basis_transfer_with_zero_balance() (gas: 33567)
[PASS] test_can_not_cancel_withdraw_adapted() (gas: 490361)
[PASS] test_cancel_unfulfilled_deposit() (gas: 87974)
[PASS] test_cancel_unfulfilled_withdrawal() (gas: 326510)
[PASS] test_cancel_withdrawal_handles_underflow() (gas: 484728)
[PASS] test_cancel_withdrawal_restores_basis() (gas: 450030)
[PASS] test_cancel_withdrawal_with_redeposit_flag(bool) (runs: 256, : 260463, ~: 256803)
[PASS] test_complex_multi_epoch_scenario() (gas: 451291)
[PASS] test_consistent_nav_calculation_across_fulfillments() (gas: 3113032)
[PASS] test_decimals_returns_underlying_token_decimals() (gas: 17567)
[PASS] test_decimals_with_different_underlying_decimals() (gas: 8613997)
[PASS] test_decimals_with_non_standard_token() (gas: 2717927)
[PASS] test_deposit_full_multiple_users_same_epoch() (gas: 388056)
[PASS] test_deposit_full_single_user_epoch0() (gas: 247523)
[PASS] test_deposit_full_single_user_later_epoch() (gas: 244991)
[PASS] test_deposit_multiple_partial_fulfillments() (gas: 372724)
[PASS] test_deposit_partial_multiple_users() (gas: 379143)
[PASS] test_deposit_partial_single_user() (gas: 280831)
[PASS] test_epoch_transitions_deposits() (gas: 84064)
[PASS] test_epoch_transitions_withdrawals() (gas: 84669)
[PASS] test_exact_withdrawal_calculation() (gas: 361212)
[PASS] test_execute_from_wrong_epoch() (gas: 18834)
[PASS] test_large_numbers_precision() (gas: 239779)
[PASS] test_manage_authorization() (gas: 434000)
[PASS] test_manage_call_with_revert() (gas: 426013)
[PASS] test_manage_complex_calldata_encoding() (gas: 940496)
[PASS] test_manage_empty_calldata() (gas: 427413)
[PASS] test_manage_multiple_calls_with_calldata() (gas: 1882243)
[PASS] test_manage_preserves_msg_sender() (gas: 562912)
[PASS] test_manage_single_call_with_calldata() (gas: 594016)
```

```
[PASS] test_manage_single_call_with_return_data() (gas: 685730)
[PASS] test_manager_input_validation() (gas: 117055)
[PASS] test_mixed_partial_fulfillments_across_epochs() (gas: 470650)
[PASS] test_multiple_cancellations() (gas: 162514)
[PASS] test_multiple_users_partial_deposit_exact_numbers() (gas: 370648)
[PASS] test_multiple_zero_fulfillments_then_full_fulfillment() (gas: 371440)
[PASS] test_nav_calculation_with_complex_premium_scenarios() (gas: 310747)
[PASS] test_nav_calculation_with_negative_exposure_scenarios() (gas: 309522)
[PASS] test_normal_withdrawal_vs_redeposit_withdrawal() (gas: 456252)
[PASS] test_onlyManager_functions() (gas: 137725)
[PASS] test_onlyOwner_functions() (gas: 50575)
[PASS] test_optOutOfReDeposit_preserves_amount_and_basis() (gas: 260084)
[PASS] test_optOutOfReDeposit_user_can_change_back_to_false() (gas: 261661)
[PASS] test_optOutOfReDeposit_works_across_multiple_epochs() (gas: 362476)
[PASS] test_partial_deposit_exact_numbers() (gas: 277199)
[PASS] test_partial_deposit_share_calculation() (gas: 276677)
[PASS] test_partial_withdrawal_exact_numbers() (gas: 391439)
[PASS] test_redeposit_emits_deposit_requested_event() (gas: 352682)
[PASS] test_redeposit_execution_flow() (gas: 363256)
[PASS] test_redeposit_flag_persists_across_epochs(bool) (runs: 256, : 389098, ~: 419004)
[PASS] test_redeposit_with_performance_fee() (gas: 386722)
[PASS] test_requestWithdrawalFrom_accumulates_with_existing_withdrawal(bool) (runs: 256, : 306628, ~: 296756)
[PASS] test_requestWithdrawalFrom_all_shares(bool) (runs: 256, : 259206, ~: 249334)
[PASS] test_requestWithdrawalFrom_basic_functionality(bool) (runs: 256, : 305673, ~: 314924)
[PASS] test_requestWithdrawalFrom_emits_event(bool) (runs: 256, : 293182, ~: 302433)
[PASS] test_requestWithdrawalFrom_multiple_users(bool,bool) (runs: 256, : 492536, ~: 490982)
[PASS] test_requestWithdrawalFrom_only_manager(bool,bool) (runs: 256, : 300574, ~: 310447)
[PASS] test_requestWithdrawalFrom_sets_redeposit_flag(bool) (runs: 256, : 296436, ~: 305765)
[PASS] test_requestWithdrawalFrom_zero_shares(bool) (runs: 256, : 259597, ~: 250969)
[PASS] test_requestWithdrawal_sets_no_redeposit_flag() (gas: 283624)
[PASS] test_reserved_withdrawal_assets_tracking() (gas: 381822)
[PASS] test_sequential_deposit_withdrawal_cycles() (gas: 457600)
[PASS] test_shouldReDeposit_remains_true_after_additional_withdrawal() (gas: 330747)
[PASS] test_total_supply_updates() (gas: 413691)
[PASS] test_vaultParameters() (gas: 39906)
[PASS] test_vault_can_receive_batch_erc1155_tokens() (gas: 719024)
[PASS] test_vault_can_receive_erc1155_tokens() (gas: 644828)
[PASS] test_vault_can_receive_erc721_tokens() (gas: 668636)
[PASS] test_vault_can_receive_eth() (gas: 21506)
[PASS] test_vault_can_receive_eth_from_manager() (gas: 21706)
[PASS] test_vault_can_receive_multiple_erc721_tokens() (gas: 744566)
[PASS] test_withdrawal_not_fulfillable() (gas: 359611)
[PASS] test_withdrawal_partial() (gas: 499408)
[PASS] test_withdrawal_scenario_verification() (gas: 363821)
[PASS] test_withdrawal_with_loss_no_performance_fee() (gas: 461290)
[PASS] test_withdrawal_with_multiple_users() (gas: 462158)
[PASS] test_withdrawal_with_profit_performance_fee() (gas: 416308)
[PASS] test_zero_amount_deposit() (gas: 38340)
[PASS] test_zero_fulfillment_deposits_with_pending_deposits() (gas: 217788)
[PASS] test_zero_fulfillment_with_multiple_users() (gas: 451729)
[PASS] test_zero_fulfillment_withdrawals_with_pending_withdrawals() (gas: 350467)
Suite result: ok. 92 passed; 0 failed; 0 skipped; finished in 106.45ms (410.73ms CPU time)
```

```
Ran 60 tests for test/PanopticVaultAccountant.t.sol:PanopticVaultAccountantTest
[PASS] testFuzz_computeNAV_priceDeviationBoundary(int24) (runs: 256, : 633083, ~: 633096)
[PASS] testFuzz_computeNAV_tokenBalances(uint128,uint128,uint128) (runs: 256, : 553003, ~: 553793)
[PASS] testFuzz_computeNAV_underlyingBalance(uint256) (runs: 256, : 71360, ~: 71827)
[PASS] testFuzz_updatePoolsHash(address,bytes32) (runs: 256, : 37073, ~: 37073)
[PASS] test_computeNAV_basicScenario() (gas: 636275)
[PASS] test_computeNAV_emptyPools() (gas: 71912)
[PASS] test_computeNAV_ethHandling() (gas: 632666)
[PASS] test_computeNAV_exactCalculation_negativeExposureToZero() (gas: 617059)
[PASS] test_computeNAV_exactCalculation_noPositions() (gas: 632245)
[PASS] test_computeNAV_exactCalculation_onlyUnderlyingToken() (gas: 339425)
[PASS] test_computeNAV_exactCalculation_sameTokenAsUnderlying() (gas: 486182)
[PASS] test_computeNAV_exactCalculation_singleLegPosition() (gas: 1119179)
[PASS] test_computeNAV_exactCalculation_withExactPremiums() (gas: 426014)
[PASS] test_computeNAV_exactCalculation_withPremiums() (gas: 674604)
[PASS] test_computeNAV_flippedOracles_conversion() (gas: 820686)
[PASS] test_computeNAV_flippedTokens() (gas: 633489)
[PASS] test_computeNAV_fullIntegration_realScenario() (gas: 1206693)
[PASS] test_computeNAV_inRangeCall_long() (gas: 776050)
[PASS] test_computeNAV_inRangePut_long() (gas: 775371)
```



```
[PASS] test_computeNAV_incorrectPositionList_wrongLegsCount() (gas: 288178)
[PASS] test_computeNAV_incorrectPositionList_zeroBalance() (gas: 243664)
[PASS] test_computeNAV_invalidPools() (gas: 62114)
[PASS] test_computeNAV_ironCondor_fourLegPosition() (gas: 800907)
[PASS] test_computeNAV_largeNumbers_noOverflow() (gas: 685636)
[PASS] test_computeNAV_maxPriceDeviationBoundary() (gas: 632073)
[PASS] test_computeNAV_maxPriceDeviationBoundary_exact() (gas: 632191)
[PASS] test_computeNAV_multiLegPosition() (gas: 780688)
[PASS] test_computeNAV_multiPool_exactAggregation() (gas: 965269)
[PASS] test_computeNAV_multiplePoolsScenario() (gas: 964879)
[PASS] test_computeNAV_multiplePositions_aggregation() (gas: 841684)
[PASS] test_computeNAV_negativeExposure_handledCorrectly() (gas: 527922)
[PASS] test_computeNAV_negativePnL() (gas: 527488)
[PASS] test_computeNAV_outOfRangeCall_short() (gas: 774065)
[PASS] test_computeNAV_outOfRangePut_short() (gas: 773864)
[PASS] test_computeNAV_precisionTest_smallAmounts() (gas: 671732)
[PASS] test_computeNAV_priceConversion_exactRatio() (gas: 809805)
[PASS] test_computeNAV_priceDeviation_exactBoundary() (gas: 1305256)
[PASS] test_computeNAV_revert_incorrectLegsCount() (gas: 526185)
[PASS] test_computeNAV_revert_invalidPoolsHash() (gas: 62356)
[PASS] test_computeNAV_revert_stalePoolPrice() (gas: 379164)
[PASS] test_computeNAV_revert_zeroPositionBalance() (gas: 453425)
[PASS] test_computeNAV_separate_token_and_position_exposures() (gas: 578431)
[PASS] test_computeNAV_staleOraclePrice_pool() (gas: 182376)
[PASS] test_computeNAV_staleOraclePrice_token0() (gas: 508646)
[PASS] test_computeNAV_staleOraclePrice_token1() (gas: 627654)
[PASS] test_computeNAV_straddlePosition_exactCalculation() (gas: 786726)
[PASS] test_computeNAV_tokenBalances_not_netted_against_negative_collateral() (gas: 656190)
[PASS] test_computeNAV_tokenConversion() (gas: 631651)
[PASS] test_computeNAV_token_balances_preserved_despite_negative_positions() (gas: 918991)
[PASS] test_computeNAV_token_netting_issue_demonstration() (gas: 965029)
[PASS] test_computeNAV_twapCalculation_verification() (gas: 755318)
[PASS] test_computeNAV_withPositions() (gas: 774276)
[PASS] test_computeNAV_zeroAmounts_edgeCase() (gas: 491919)
[PASS] test_computeNAV_zeroBalances() (gas: 491215)
[PASS] test_lockVault_onlyOwner() (gas: 15984)
[PASS] test_lockVault_permanent() (gas: 37007)
[PASS] test_lockVault_success() (gas: 38896)
[PASS] test_updatePoolsHash_onlyOwner() (gas: 16442)
[PASS] test_updatePoolsHash_success() (gas: 40167)
[PASS] test_updatePoolsHash_vaultLocked() (gas: 62947)
Suite result: ok. 60 passed; 0 failed; 0 skipped; finished in 621.55ms (705.51ms CPU time)

Ran 3 test suites in 629.03ms (739.50ms CPU time): 175 tests passed, 0 failed, 0 skipped (175 total tests)
```


9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.